

Simulador de Sistema de Arquivos

Integrantes: Igor Praciano Thomaz e Rafael Lima Cacau

Parte 1 - Introdução ao Sistema de Arquivos com Journaling

O que é um sistema de arquivos

Um sistema de arquivos é a forma usada pelo sistema operacional para organizar dados em um dispositivo de armazenamento. Ele permite criar arquivos, agrupar esses arquivos em diretórios, acessar conteúdos por caminhos e controlar operações como cópia, remoção e renomeação.

Neste projeto, o sistema de arquivos é simulado em Java. Em vez de gravar arquivos reais em várias pastas do computador, o programa mantém uma árvore de diretórios e arquivos dentro da aplicação.

Importância

Sem um sistema de arquivos, os dados ficariam armazenados apenas como blocos sem uma organização simples para o usuário. Com ele, é possível localizar arquivos por nome, separar conteúdos em diretórios e manter uma estrutura mais fácil de consultar.

No simulador, essa ideia aparece na raiz `/`, nos diretórios criados pelo usuário e nos arquivos guardados dentro de cada diretório.

Conceito de journaling

Journaling é uma técnica usada para registrar operações importantes em um log. A ideia é manter um histórico das ações realizadas, como criação, remoção, cópia e renomeação.

Em sistemas reais, o journal pode ajudar na recuperação depois de uma falha. Neste projeto, o journal é mais simples: ele registra as operações executadas e seus status no arquivo `journal.log`.

Tipos de journaling

- **Write-Ahead Logging (WAL):** registra a operação no log antes de aplicar a alteração principal. Isso ajuda a evitar inconsistências em caso de falha.
- **Journaling de metadados:** registra principalmente alterações de estrutura, como criação ou remoção de arquivos e diretórios.
- **Log-structured file system:** organiza as gravações como uma sequência de registros em log.

O simulador usa uma forma simples de journaling de operações. Ele não implementa recuperação automática, mas mantém um registro persistente do que aconteceu.

Parte 2 - Arquitetura do Simulador

Estruturas de dados utilizadas

O projeto usa uma estrutura em árvore para representar o sistema de arquivos:

- `SimDirectory` representa um diretório.
- `SimFile` representa um arquivo.
- Cada diretório guarda seus arquivos em um `Map<String, SimFile>`.
- Cada diretório guarda seus subdiretórios em um `Map<String, SimDirectory>`.

Foi usado `LinkedHashMap` para manter a ordem de inserção dos arquivos e diretórios durante a listagem.

Representação de arquivos e diretórios

A raiz do sistema é representada pelo diretório `/`. A partir dela, o programa navega pelos caminhos informados pelo usuário.

Exemplo:

```
/documentos/arquivo.txt
```

Nesse caminho, `documentos` é um diretório dentro da raiz, e `arquivo.txt` é um arquivo dentro de `documentos`.

A classe `SimFile` armazena:

- nome do arquivo;
- conteúdo;
- data de criação;
- data de atualização.

A classe `SimDirectory` armazena:

- nome do diretório;
- arquivos do diretório;
- subdiretórios;
- data de criação.

Implementação do journaling

O journaling foi implementado na classe `Journal`. Ela grava as operações no arquivo `journal.log`.

Cada operação registrada possui:

- data e hora;
- nome da operação;
- caminho usado;
- status da operação.

Estrutura do log e operações registradas

O formato de cada linha do journal é:

```
AAAA-MM-DD HH:MM:SS | OPERACAO | CAMINHO | STATUS
```

Exemplo:

```
2026-05-28 17:52:32 | CREATE_DIRECTORY | /documentos | SUCCESS
```

Operações registradas pelo programa:

- `CREATE_DIRECTORY`
- `DELETE_DIRECTORY`
- `RENAME_DIRECTORY`
- `CREATE_FILE`
- `COPY_FILE`
- `DELETE_FILE`
- `RENAME_FILE`

O status pode ser `SUCCESS` ou `FAIL`.

Parte 3 - Implementação em Java

Classe `FileSystemSimulator`

É a classe principal do projeto. Ela inicializa o simulador, carrega o estado salvo, executa os comandos do shell e chama os métodos de manipulação.

Métodos principais:

- `createDirectory(String path)`
- `deleteDirectory(String path)`
- `renameDirectory(String path, String newName)`
- `createFile(String path, String content)`
- `copyFile(String sourcePath, String destinationPath)`
- `deleteFile(String path)`
- `renameFile(String path, String newName)`
- `listDirectory(String path)`

Também existem métodos auxiliares para separar caminhos, encontrar diretórios e validar nomes.

Classe `SimFile`

Representa um arquivo do sistema simulado. Ela guarda o nome, o conteúdo e as datas de criação e atualização.

Também possui métodos para renomear o arquivo e criar uma cópia com outro nome.

Classe `SimDirectory`

Representa um diretório do sistema simulado. Ela guarda os arquivos e subdiretórios usando mapas.

Essa classe permite montar a árvore de diretórios a partir da raiz `/`.

Classe `Journal`

Responsável por escrever o histórico de operações em `journal.log` e ler as últimas entradas do log quando o programa é iniciado.

Métodos implementados

O simulador possui métodos para:

- criar arquivos;
- copiar arquivos;
- apagar arquivos;
- renomear arquivos;
- criar diretórios;
- apagar diretórios;
- renomear diretórios;
- listar arquivos e subdiretórios.

Parte 4 - Instalação e funcionamento

Dependências

É necessário ter o Java instalado. O projeto não usa bibliotecas externas.

Estrutura do projeto

```
src/  
  FileSystemSimulator.java  
  Journal.java  
  SimDirectory.java  
  SimFile.java  
filesystem.dat  
journal.log  
README.md
```

O arquivo `filesystem.dat` armazena o estado serializado do sistema de arquivos. O arquivo `journal.log` guarda o histórico das operações.

Como compilar

No Linux ou macOS:

```
javac -d out src/*.java
```

No Windows PowerShell:

```
javac -d out src\*.java
```

Como executar

No Linux, macOS ou Windows:

```
java -cp out FileSystemSimulator
```

Comandos disponíveis

```
mkdir /diretorio
rmdir /diretorio
touch /diretorio/arquivo.txt "conteudo"
cp /origem/arquivo.txt /destino/arquivo.txt
rename /caminho/atual novo_nome
rm /diretorio/arquivo.txt
ls /diretorio
help
exit
```

Exemplos de uso

```
mkdir /documentos
touch /documentos/arquivo.txt "conteudo do arquivo"
ls /documentos
cp /documentos/arquivo.txt /backup.txt
rename /documentos/arquivo.txt novo_nome.txt
rm /documentos/novo_nome.txt
rmdir /documentos
exit
```

Observações sobre funcionamento

O shell é simples e usa caminhos absolutos, como `/documentos` ou `/documentos/arquivo.txt`.

O diretório só pode ser removido quando está vazio. Para remover um diretório com arquivos, primeiro é necessário apagar os arquivos dentro dele.

O journal registra as operações, mas não refaz automaticamente operações depois de uma falha. A persistência principal do simulador é feita pelo arquivo `filesystem.dat`.

Repositório: [rcacau/gerenciadorArquivos](#)